Bootstrapping is All You Need: Secure Transformer Inference via Improved CKKS Functional Bootstrapping

Pan Xiao, Rending Ouyang, Heng Zhang, Jiawen Zhang, Jian Liu ✉

The State Key Laboratory of Blockchain and Data Security, Zhejiang University
{pan.xiao, oyyy, 12521213, kevinzh, jian.liu}@zju.edu.cn

***Abstract*—Fully homomorphic encryption (FHE) enables non-interactive secure transformer inference (NISTI). Due to the high cost of bootstrapping, conventional approaches typically choose parameters that support a large multiplicative depth to reduce bootstrapping frequency. However, larger depth directly increases ciphertext size, resulting in higher communication and computation overheads.**

In this paper, we introduce a novel functional bootstrapping (FBS) scheme that fundamentally reshapes the computation paradigm for NISTI: by fusing as many operations as possible into each bootstrapping operation, our approach significantly reduces the prescribed multiplicative depth.

Our FBS achieves a trigonometric minimax approximation for the target function, making it well suited for precision-sensitive components such as transformer nonlinear layers. Furthermore, we incorporate linear layers into the slot-to-coefficient (S2C) transformation within FBS, thereby eliminating the need to evaluate them separately. Building on these innovations, we present a complete NISTI framework that achieves a 1.9× speedup in runtime (from 662.3s to 349.5s) and a 3× reduction in communication (from 48.3MB to 16.1MB) compared with the state-of-the-art.

1. Introduction

Fully homomorphic encryption (FHE) enables arbitrary computations to be performed directly on encrypted data [21]. It serves as a key primitive for *non-interactive secure transformer inference* (NISTI) [42], [37], [43]: the client encrypts its input under FHE and sends it to the server, which executes the transformer model over the encrypted input and returns the encrypted output. Compared with interactive protocols [27], [25], [38], [26], [33], such approaches offer several advantages: (1) reduced communication overhead, (2) eliminating the need for continuous client involvement, and (3) improved utilization of hardware acceleration.

The FHE scheme commonly adopted for NISTI is CKKS [16], [17], a *leveled* FHE scheme that supports computations up to a *prescribed* multiplicative depth. During homomorphic evaluation, this computation budget is gradually depleted; once it becomes insufficient, a *bootstrapping* operation is required to refresh the ciphertext and restore its computation budget. Bootstrapping is widely regarded as the main bottleneck in NISTI; for example, in the state-of-the-art NISTI [43], it accounts for 66.8% of the total runtime. To reduce bootstrapping frequency, conventional approaches typically choose parameters that support a large multiplicative depth, which in turn increases ciphertext size, leading to higher communication and computation overheads.

In this paper, we introduce a novel *functional bootstrapping* (FBS) scheme that fundamentally reshapes the NISTI computation paradigm: by fusing as many operations as possible into each bootstrapping operation, it substantially reduces the prescribed multiplicative depth.

FBS with improved precision. The state-of-the-art FBS [5] approximates a periodic *target function* using Fourier series, which achieves an optimal trigonometric approximation under the L_2 norm, but does not necessarily minimize the worst-case error (i.e., the L_∞ norm) – a more relevant performance metric in practice. The classical Remez algorithm minimizes the L_∞ norm, but it is restricted to a polynomial basis and cannot be directly adapted to the trigonometric basis required by the periodic structure of FBS. In this paper, we establish a theoretical foundation for trigonometric *minimax approximation* in FBS. Specifically, we prove the existence of a trigonometric minimax approximation for a given target function (Theorem 3.4). Based on this result, we develop a trigonometric Remez algorithm that efficiently computes the approximation.

Fusing linear layers into FBS. In transformer models, linear and nonlinear layers are typically alternating. FBS can naturally incorporate nonlinear layers into the bootstrapping process, whereas linear layers have to be evaluated separately, introducing additional computational overhead and level consumption. In this paper, we show that, by exploiting the closure property of linear maps under composition, arbitrary linear transformations can be fused directly into *slot-to-coefficient* (S2C) operations. Specifically, for linear mappings of the form $\mathbf{y} = \mathbf{x}\mathbf{W}$ and affine transformations of the form $\mathbf{y} = \mathbf{x}\mathbf{W} + \mathbf{b}$, we precompute equivalent fused transformations and evaluate them jointly within FBS. This *functional S2C* approach eliminates the need for separate

✉ Jian Liu is the corresponding author.

homomorphic evaluation of linear layers and reduces the overall computational overhead and level consumption.

The NISTI framework. We introduce a new NISTI framework that fuses as many operations as possible into each FBS operation, thereby minimizing the prescribed multiplicative depth and substantially reducing both communication and computation overheads. We further propose a batch-S2C technique for simultaneously processing multiple FBS instances, significantly reducing the number of homomorphic operations. In addition, we integrate complementary optimizations, including interleaved packing [43], lazy key switching [8], and hoisting [24] to further improve overall efficiency. We provide a full-fledged implementation and comprehensive evaluation of the proposed framework. For a BERT-based model, our system performs end-to-end inference in 349.5 seconds while requiring only 16.1MB of communication per query, amortized over 256 inputs. Compared with the state-of-the-art [43], it delivers a 1.9× speedup in runtime and a 3× reduction in communication.

We summarize our contributions as follows:

- A new NISTI paradigm that fuses as many operations as possible into each bootstrapping operation, thereby reducing the prescribed multiplicative depth.
- A new FBS scheme that achieves a trigonometric minimax approximation for a target function, establishing a theoretical foundation for FBS minimax approximation.
- A new technique for incorporating linear layers into the S2C transformation within FBS, along with a batch-S2C optimization for efficient parallel processing.
- A complete NISTI framework that achieves a 1.9× speedup in runtime and a 3× reduction in communication compared with the state-of-the-art.

2. Preliminaries

In this section, we provide the necessary preliminaries for this paper.

2.1. Notation

We use $[n]$ to denote the set $\{0, 1, \ldots, n-1\}$. Bold lowercase letters (e.g., $\mathbf{x} \in \mathbb{R}^{1\times n}$) denote vectors, and bold uppercase letters (e.g., $\mathbf{X} \in \mathbb{R}^{n\times m}$) denote matrices. For a matrix $\mathbf{X}$, $\mathbf{X}[i,:]$ and $\mathbf{X}[:,j]$ denote its i-th row and j-th column, respectively. Table 1 summarizes the notation used throughout the paper.

2.2. Transformer

Figure 1 shows the structure of a transformer. It takes a matrix $\mathbf{X} \in \mathbb{R}^{n'\times m}$ and runs as follows:

1) For each head $h \in [H]$, given $\mathbf{W}_{Q,h}, \mathbf{W}_{K,h}, \mathbf{W}_{V,h} \in \mathbb{R}^{m\times d}$, it computes

$$\mathbf{Q}_h||\mathbf{K}_h||\mathbf{V}_h = \mathbf{X}\cdot(\mathbf{W}_{Q,h}||\mathbf{W}_{K,h}||\mathbf{W}_{V,h}) \in \mathbb{R}^{n'\times(3d)}.$$

Table 1: Frequently used notations.

Notation	Description
N	polynomial degree in RNS-CKKS
n	# SIMD slots, $n = N/2$
$\mathsf{E}()$	encryption of a polynomial
$\mathsf{Enc}()$	SIMD encryption of a vector
$\widetilde{\mathbf{x}}$	an SIMD ciphertext of a vector
$\widetilde{\mathbf{X}}^{\mathsf{row}}$	SIMD ciphertexts of a matrix in row-packing
$\widetilde{\mathbf{X}}^{\mathsf{col}}$	SIMD ciphertexts of a matrix in column-packing
$\widetilde{\mathbf{X}}^{\mathsf{diag}}$	SIMD ciphertexts of a matrix in diagonal-packing
$\mathsf{Rot}_s(\widetilde{\mathbf{x}})$	left-rotates $\mathbf{m}$ by s slots
$\mathbf{U}_0$	DFT matrix

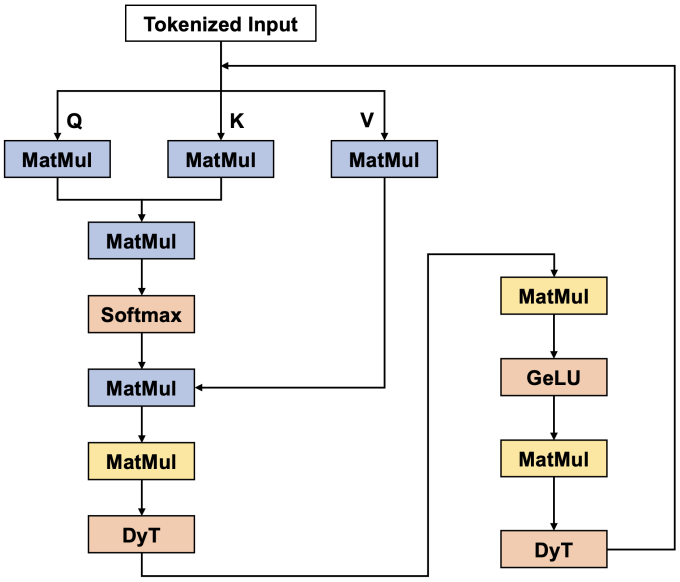


Figure 1: Structure of a transformer.

2) For each head $h \in [H]$, it computes

$$\mathbf{A}_h = \mathbf{Q}_h \cdot \mathbf{K}_h^\top \in \mathbb{R}^{n'\times n'}.$$

3) For each head $h \in [H]$, it computes

$$\mathbf{B}_h = \mathsf{Softmax}\left(\frac{\mathbf{A}_h}{\sqrt{d}}\right) \in \mathbb{R}^{n'\times n'},$$

where $\mathsf{Softmax}(\mathbf{x})[i] = \frac{e^{\mathbf{x}[i]}}{\sum_{i=1}^{n} e^{\mathbf{x}[i]}}$.

4) For each head $h \in [H]$, it computes

$$\mathbf{C}_h = \mathbf{B}_h \cdot \mathbf{V}_h \in \mathbb{R}^{n'\times d}.$$

5) The outputs of all heads are concatenated and projected by $\mathbf{W}_0 \in \mathbb{R}^{(d\cdot H)\times m}$:

$$\mathbf{T} = (\mathbf{C}_0||\ldots||\mathbf{C}_{H-1})\cdot\mathbf{W}_0 \in \mathbb{R}^{n'\times m},$$

and each value in $\mathbf{T}$ is then normalized. Recent

work [44] in the machine learning community has shown that a *dynamic tanh* function can be used for normalization:

$$y = \alpha \cdot \tanh(\beta x) + \gamma,$$

where α, β, γ are learnable parameters. We summarize this step as

$$\mathbf{D} = \alpha \cdot \tanh\big(\beta \cdot (\mathbf{C}_0 || \ldots || \mathbf{C}_{H-1}) \cdot \mathbf{W}_0\big) + \gamma \in \mathbb{R}^{n' \times m}.$$

6) Given $\mathbf{W}_1 \in \mathbb{R}^{m \times k}$ and $\mathbf{b}_1 \in \mathbb{R}^{n' \times k}$, it computes

$$\mathbf{E} = \mathsf{GELU}\big(\mathbf{D} \cdot \mathbf{W}_1 + \mathbf{b}_1\big) \in \mathbb{R}^{n' \times k},$$

where

$$\mathsf{GELU}(x) = 0.5x(1 + \tanh\left[\sqrt{2/\pi}(x + 0.044715x^3)\right]).$$

7) Given $\mathbf{W}_2 \in \mathbb{R}^{k \times m}$ and $\mathbf{b}_2 \in \mathbb{R}^{n' \times m}$, it computes

$$\mathbf{X} = \alpha \cdot \tanh\big(\beta \cdot (\mathbf{E} \cdot \mathbf{W}_2 + \mathbf{b}_2)\big) + \gamma,$$

which is then input into Step 1 for another iteration.

2.3. Fully homomorphic encryption

The FHE scheme employed in this work is the full *residue number system* (RNS) variant of Cheon-Kim-Kim-Song (CKKS) [16], [17], which is a *leveled* FHE that supports computations up to a prescribed multiplicative depth L. In RNS-CKKS, both the plaintexts and ciphertexts are represented as elements of the polynomial ring:

$$\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N + 1),$$

where $Q = \Pi_{i=0}^{L} q_i$ with distinct primes q_i. After a multiplication (or certain operations), the noise grows; to control noise, the scheme rescales by dividing by one of the primes, say q_L; the new modulus becomes $\Pi_{i=0}^{L-1} q_i$. Each rescaling reduces the level by one. When a ciphertext's level becomes too low to support further computations, a *bootstrapping* operation is performed to refresh it back to Q.

Slot encoding and SIMD. RNS-CKKS supports *single instruction multiple data* (SIMD), allowing a vector $\mathbf{m} \in \mathbb{R}^{1 \times n}$ (with $n = N/2$) to be encrypted into a single ciphertext, and thereby enabling batch processing of n encrypted elements without introducing additional computational cost. It achieves this through *slot encoding*, which encodes $\mathbf{m}$ into a polynomial in $\mathcal{R}_Q$ before encryption. More concretely, let $\zeta = \exp(\frac{\pi\sqrt{-1}}{N})$ be a $(2N)$-th primitive root of unity, define $\zeta_j = \zeta^{5^j} \ \forall j \in [n]$ and

$$\mathbf{U} = \begin{bmatrix} 1 & \zeta_0 & \zeta_0^2 & \cdots & \zeta_0^{N-1} \\ 1 & \zeta_1 & \zeta_1^2 & \cdots & \zeta_1^{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{n-1} & \zeta_{n-1}^2 & \cdots & \zeta_{n-1}^{N-1} \end{bmatrix} \in \mathbb{C}^{n \times N}.$$

The vector $\mathbf{m}$ is encoded as a polynomial $\sum_{i=0}^{N-1} e_i \cdot X^i$ with

$$\mathbf{e} = (e_i)_{0 \le i < N} = \mathsf{IDFT}(\mathbf{m}) = \frac{1}{N}\left(\mathbf{m} \cdot \mathbf{U}^* + \mathbf{m}^* \cdot \mathbf{U}\right),$$

where $\mathbf{U}^*$ is the conjugate of $\mathbf{U}$. Decoding is the reverse process:

$$\mathbf{m} = \mathsf{DFT}(\mathbf{e}) = \mathbf{e} \cdot \mathbf{U}^\top.$$

Throughout this paper, we use $\mathsf{E}()$ to denote the encryption of a polynomial and use $\mathsf{Enc}()$ to denote the SIMD encryption of a vector. An SIMD ciphertext is written as $\widetilde{\mathbf{m}} = \mathsf{Enc}(\mathbf{m})$. Slightly abusing notation, we use $+, -, \cdot$ to represent operations on both plaintexts and ciphertexts. Additionally, SIMD ciphertexts support rotations: $\mathsf{Rot}_s(\widetilde{\mathbf{m}})$ left-rotates the vector $\mathbf{m}$ by s slots.

Bootstrapping. RNS-CKKS bootstrapping refreshes the ciphertext modulus from q_0 back to Q, allowing further homomorphic operations. The bootstrapping workflow primarily comes in two variants; this paper focuses on the "S2C-first" variant, which runs as follows:

1) S2C: This operation turns $\mathsf{Enc}(\mathbf{m})$ into $\mathsf{E}\big(m(X)\big)$, where $m(X) \in \mathcal{R}_{q_0}$ is a polynomial whose coefficients are $\mathbf{m}$. Suppose the corresponding slot values of $\mathsf{E}\big(m(X)\big)$ are $\mathbf{z}$, then we have $\mathbf{z} = \mathsf{DFT}(\mathbf{m})$. That means we only need to homomorphically evaluate DFT on $\mathsf{Enc}(\mathbf{m})$:

$$\mathsf{Enc}\big(\mathsf{DFT}(\mathbf{m})\big) = \mathsf{Enc}(\mathbf{m} \cdot \mathbf{U}_0^\top) = \mathsf{Enc}(\mathbf{z}) = \mathsf{E}\big(m(X)\big),$$

where

$$\mathbf{U}_0 = \begin{bmatrix} 1 & \zeta_0^2 & \zeta_0^4 & \cdots & \zeta_0^{N-2} \\ 1 & \zeta_1^2 & \zeta_1^4 & \cdots & \zeta_1^{N-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{n-1}^2 & \zeta_{n-1}^4 & \cdots & \zeta_{n-1}^{N-2} \end{bmatrix} \in \mathbb{C}^{n \times n}.$$

2) ModRaise: This operation lifts $\mathsf{E}(m(X))$ from modulus q_0 to Q, yielding a ciphertext that decrypts to $m(X) + q_0 I(X) \in \mathcal{R}_Q$, where $q_0 I(X)$ has coefficients that are small multiples of q_0.

3) C2S: This operation turns $\mathsf{E}\big(m(X) + q_0 I(X)\big)$ into $\mathsf{Enc}(\mathbf{m} + q_0 \mathbf{I})$. Suppose the corresponding slot values of $\mathsf{E}\big(m(X) + q_0 I(X)\big)$ are $\mathbf{z}'$, we have $\mathbf{m} + q_0 \mathbf{I} = \mathsf{IDFT}(\mathbf{z}')$. That means we only need to homomorphically evaluate IDFT on $\mathsf{Enc}(\mathbf{z}')$.

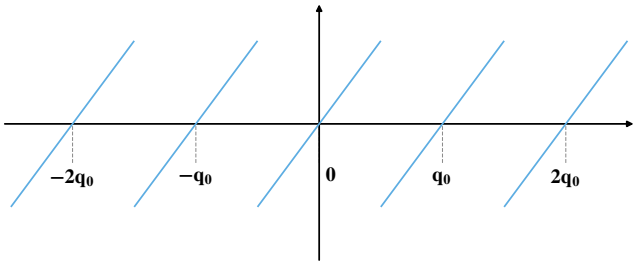
$$\mathsf{Enc}\big(\mathsf{IDFT}(\mathbf{z}')\big) = \mathsf{Enc}\big(\mathbf{z}'(\mathbf{U}_0^\top)^{-1}\big) = \mathsf{Enc}(\mathbf{m} + q_0 \mathbf{I}).$$

4) EvalMod: This operation homomorphically evaluates $\mathsf{Enc}(\mathbf{m} + q_0 \mathbf{I})$ modulo q_0, removing the redundant term $q_0 \mathbf{I}$ and yielding $\widetilde{\mathbf{m}} = \mathsf{Enc}(\mathbf{m})$ under modulus Q. The modular reduction, denoted by mod (cf. Figure 2(a)), can be approximated by trigonometric polynomials.

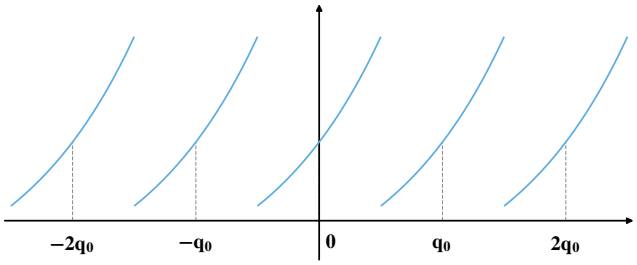
We express the bootstrapping process as:

$$\widetilde{\mathbf{m}}' = \mathsf{EvalMod} \circ \mathsf{C2S} \circ \mathsf{ModRaise} \circ \mathsf{S2C}(\widetilde{\mathbf{m}}).$$

Functional Bootstrapping. Instead of merely extracting $\mathbf{m}$ in EvalMod, *functional bootstrapping* (FBS) evaluates an arbitrary function f over each slot. To achieve this,



(a) Modular reduction (mod).



(b) Periodic exp.

Figure 2: Target functions for EvalMod.

FBS embeds f into each period of the mod function (cf. Figure 2(b), which illustrates the case $f = \exp$), yielding a periodic target function f_T. Evaluating f_T during EvalMod then simultaneously removes $q_0\mathbf{I}$ and applies f, directly producing a ciphertext, whose slots contain $f(m_i)\ \forall i \in [n]$. The state-of-the-art FBS [5] approximates f_T using Fourier series and demonstrates its effectiveness over a number of functions such as exp and GELU. We express the FBS process as:

$$\widetilde{\mathbf{m}}' = \mathsf{EvalMod}_f \circ \mathsf{C2S} \circ \mathsf{ModRaise} \circ \mathsf{S2C}(\widetilde{\mathbf{m}}).$$

2.4. Homomorphic matrix multiplication

We first introduce three packing formats that are extensively used in this paper:

- *row-packing.* For $\mathbf{X} \in \mathbb{R}^{m\times n}$, $\widetilde{\mathbf{X}}^{\mathsf{row}}$ is a set of m SIMD ciphertexts encrypting the m rows of $\mathbf{X}$:
$$\widetilde{\mathbf{X}}^{\mathsf{row}} = \left[\widetilde{\mathbf{X}}[0,:], \widetilde{\mathbf{X}}[1,:], \cdots, \widetilde{\mathbf{X}}[m-1,:]\right].$$
- *column-packing.* For $\mathbf{X} \in \mathbb{R}^{n\times m}$, $\widetilde{\mathbf{X}}^{\mathsf{col}}$ is a set of m SIMD ciphertexts encrypting the m columns of $\mathbf{X}$:
$$\widetilde{\mathbf{X}}^{\mathsf{col}} = \left[\widetilde{\mathbf{X}}[:,0], \widetilde{\mathbf{X}}[:,1], \cdots, \widetilde{\mathbf{X}}[:,m-1]\right].$$
- *diagonal-packing.* For a square matrix $\mathbf{X} \in \mathbb{R}^{n\times n}$, $\widetilde{\mathbf{X}}^{\mathsf{diag}}$ is a set of n SIMD ciphertexts encrypting the n cyclic diagonals of $\mathbf{X}$:
$$\widetilde{\mathbf{X}}^{\mathsf{diag}} = \left[\widetilde{\mathbf{X}}\langle 0\rangle, \widetilde{\mathbf{X}}\langle 1\rangle, \cdots, \widetilde{\mathbf{X}}\langle n-1\rangle\right],$$
where $\forall k \in [n]$,
$$\mathbf{X}\langle k\rangle = \left[\mathbf{X}[0, k \bmod n], \cdots, \mathbf{X}[n-1, (n-1+k) \bmod n]\right].$$

Different packing formats can be multiplied with each other as follows:

- *row-packing* $\times$ *plaintext* $\rightarrow$ *row-packing* [24]. Let $\widetilde{\mathbf{X}}^{\mathsf{row}}$ be the row-packing of $\mathbf{X} \in \mathbb{R}^{m\times n}$ and $\mathbf{W} \in \mathbb{R}^{n\times d}$:
$$\widetilde{\mathbf{Y}}^{\mathsf{row}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{row}} \cdot \mathbf{W},$$
which requires md ciphertext-plaintext multiplications and $2m\sqrt{d}$ rotations.
- *column-packing* $\times$ *plaintext* $\rightarrow$ *column-packing* [23]: Let $\widetilde{\mathbf{X}}^{\mathsf{col}}$ be the column-packing of $\mathbf{X} \in \mathbb{R}^{n\times m}$ and $\mathbf{W} \in \mathbb{R}^{m\times d}$,
$$\widetilde{\mathbf{Y}}^{\mathsf{col}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{col}} \cdot \mathbf{W},$$
which requires md ciphertext-plaintext multiplications.
- *column-packing* $\times$ *row-packing* $\rightarrow$ *diagonal-packing* [38]. Let $\widetilde{\mathbf{X}}^{\mathsf{col}}$ be the column-packing of $\mathbf{X} \in \mathbb{R}^{n\times d}$ and $\widetilde{\mathbf{Y}}^{\mathsf{row}}$ be the row-packing of $\mathbf{Y} \in \mathbb{R}^{d\times n}$:
$$\widetilde{\mathbf{Z}}^{\mathsf{diag}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{col}} \cdot \widetilde{\mathbf{Y}}^{\mathsf{row}},$$
which requires nd ciphertext-ciphertext multiplications and nd rotations.
- *diagonal-packing* $\times$ *column-packing* $\rightarrow$ *column-packing* [23]. Let $\widetilde{\mathbf{X}}^{\mathsf{diag}}$ be the diagonal-packing of $\mathbf{X} \in \mathbb{R}^{n\times n}$ and $\widetilde{\mathbf{Y}}^{\mathsf{col}}$ be the column-packing of $\mathbf{Y} \in \mathbb{R}^{n\times d}$:
$$\widetilde{\mathbf{Z}}^{\mathsf{col}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{diag}} \cdot \widetilde{\mathbf{Y}}^{\mathsf{col}},$$
which requires nd ciphertext-ciphertext multiplications and $2d\sqrt{n}$ rotations.

More details can be found in Appendix A.

2.5. Interleaved Packing

In Section 2.4, we assume that each packed row, column, or diagonal has length exactly n. In practice, however, their lengths may be much smaller than n, leading to substantial slot wastage. To address this issue, we pack rows, columns, and diagonals from multiple matrices into a single ciphertext, thereby maximizing slot utilization.

Packing formats. Assume $n'|n$, where n' is the length of vectors. Then, we can pack $B = n/n'$ vectors in a single ciphertext. The *interleaved packing* [43] of $\mathbf{v}^{(i)} = [v_0^{(i)}, v_1^{(i)}, \ldots, v_{n'-1}^{(i)}]\ \forall\ i \in [B]$ is defined as follows:

$$\begin{aligned}\mathsf{ILPack}(\mathbf{v}^{(0)}, \ldots, \mathbf{v}^{(B-1)}) = \mathsf{Enc}(\big[&v_0^{(0)}, \ldots, v_0^{(B-1)},\\ &v_1^{(0)}, \ldots, v_1^{(B-1)},\\ &\ldots,\\ &v_{n'-1}^{(0)}, \ldots, v_{n'-1}^{(B-1)}\big]).\end{aligned}$$

In this way, we can perform B rotations in a batch with a

single homomorphic rotation:

$$\mathsf{Rot}_{r\cdot B}\big(\mathsf{ILPack}(\mathbf{v}^{(0)},\dots,\mathbf{v}^{(B-1)})\big) = \mathsf{ILPack}(\mathsf{Rot}_r(\mathbf{v}^{(0)}),\dots,\mathsf{Rot}_r(\mathbf{v}^{(B-1)})).$$

Interleaved matrix multiplication. The interleaved packing is compatible with the matrix multiplication methods described in Section 2.4.

- *row-packing* $\times$ *plaintext* $\to$ *row-packing.* Suppose we have $B = n/n'$ row-packed matrices: $\mathbf{X}^{(i)} \in \mathbb{R}^{m\times n'}$ $\forall i \in [B]$. Given a plaintext matrix $\mathbf{W} \in \mathbb{R}^{n'\times n'}$, we can compute $\mathbf{Y}^{(i)} = \mathbf{X}^{(i)} \cdot \mathbf{W}$ $\forall i \in [B]$ in a batch:

  $$\mathsf{ILPack}^{\mathsf{row}}(\mathbf{Y}^{(0)},\dots,\mathbf{Y}^{(B-1)}) = \mathsf{ILPack}^{\mathsf{row}}(\mathbf{X}^{(0)},\dots,\mathbf{X}^{(B-1)})\cdot\tau(\mathbf{W})$$

  where $\tau(\mathbf{W}) \in \mathbb{R}^{n\times n}$ and $\forall\ i,j \in [n]$

  $$\tau(\mathbf{W})_{i,j} = \begin{cases} \mathbf{W}_{\lfloor i/B\rfloor,\lfloor j/B\rfloor} & \text{if } i \equiv j \pmod B \\ 0 & \text{else} \end{cases}.$$

  For a non-square matrix $\mathbf{W} \in \mathbb{R}^{n'\times d}$ where d is a multiple of n',[1] we could split $\mathbf{W}$ into d/n' matrices and process them separately, as above.
- *column-packing* $\times$ *plaintext* $\to$ *column-packing.* Suppose we have $B = n/n'$ column-packed matrices $\mathbf{X}^{(i)} \in \mathbb{R}^{n'\times m}$ $\forall i \in [B]$. Given a plaintext matrix $\mathbf{W} \in \mathbb{R}^{m\times d}$, we can compute $\mathbf{Y}^{(i)} = \mathbf{X}^{(i)}\cdot\mathbf{W}$ $\forall i \in [B]$ in a batch:

  $$\mathsf{ILPack}^{\mathsf{col}}(\mathbf{Y}^{(0)},\dots,\mathbf{Y}^{(B-1)}) = \mathsf{ILPack}^{\mathsf{col}}(\mathbf{X}^{(0)},\dots,\mathbf{X}^{(B-1)})\cdot\mathbf{W}.$$

- *column-packing* $\times$ *row-packing* $\to$ *diagonal-packing.* Suppose we have $B = n/n'$ column-packed matrices $\mathbf{X}^{(i)} \in \mathbb{R}^{n'\times d}$ $\forall i \in [B]$ and row-packed matrices $\mathbf{Y}^{(i)} \in \mathbb{R}^{d\times n'}$ $\forall i \in [B]$. We can compute $\mathbf{Z}^{(i)} = \mathbf{X}^{(i)}\cdot\mathbf{Y}^{(i)}$ $\forall i \in [B]$ in a batch:

  $$\mathsf{ILPack}^{\mathsf{diag}}(\mathbf{Z}^{(0)},\dots,\mathbf{Z}^{(B-1)}) = \mathsf{ILPack}^{\mathsf{col}}(\mathbf{X}^{(0)},\dots,\mathbf{X}^{(B-1)})\cdot\mathsf{ILPack}^{\mathsf{row}}(\mathbf{Y}^{(0)},\dots,\mathbf{Y}^{(B-1)}).$$

- *diagonal-packing* $\times$ *column-packing* $\to$ *column-packing.* Suppose we have $B = n/n'$ diagonal-packed matrices $\mathbf{X}^{(i)} \in \mathbb{R}^{n'\times n'}$ $\forall i \in [B]$ and column-packed matrices $\mathbf{Y}^{(i)} \in \mathbb{R}^{n'\times d}$ $\forall i \in [B]$. We can compute $\mathbf{Z}^{(i)} = \mathbf{X}^{(i)}\cdot\mathbf{Y}^{(i)}$ $\forall i \in [B]$ in a batch:

  $$\mathsf{ILPack}^{\mathsf{col}}(\mathbf{Z}^{(0)},\dots,\mathbf{Z}^{(B-1)}) = \mathsf{ILPack}^{\mathsf{diag}}(\mathbf{X}^{(0)},\dots,\mathbf{X}^{(B-1)})\cdot\mathsf{ILPack}^{\mathsf{col}}(\mathbf{Y}^{(0)},\dots,\mathbf{Y}^{(B-1)}).$$

3. FBS with Improved Precision

To approximate a periodic target function f_T (cf. Figure 2(b)), a trigonometric basis is particularly well suited, as it provides an efficient approximation over a single period while naturally extending to a global periodic representation.

1. In a transformer, $n'|d$ always holds.

The state-of-the-art FBS [5] uses Fourier series for this purpose. While Fourier series provides an optimal trigonometric approximation under the L_2 norm, it does not necessarily minimize the worst-case approximation error (i.e., the L_∞ norm), which is often the more relevant performance metric in practice. On the other hand, the Remez algorithm achieves a minimax approximation by minimizing the L_∞ norm, yielding a more uniform approximation with smaller peak error than Fourier series. However, the classical Remez algorithm is restricted to a polynomial basis and cannot directly use the trigonometric basis to accommodate the periodic structure required by FBS. In this section, we extend the classical Remez algorithm to the trigonometric basis and prove that the resulting approximation is minimax optimal among all trigonometric approximations of the same degree.

Let

$$\mathcal{T}_d = \left\{ p(x) = c_0 + \sum_{m=1}^{d} \big(a_m\cos(m\eta x) + b_m\sin(m\eta x)\big) \right\},$$

where $\eta = \frac{2\pi}{q}$, denote the space of trigonometric polynomials of degree at most d. For a fixed interval $[\alpha,\beta] \subseteq [-\frac{q}{2},\frac{q}{2}]$,[2] we seek the minimax approximation:

$$\min_{p\in\mathcal{T}_d} \|f_T - p\|_\infty.$$

A trigonometric polynomial $p \in \mathcal{T}_d$ achieving this minimum is called a *degree-d trigonometric minimax approximation* of f_T on $[\alpha,\beta]$. We first prove that such a polynomial exists and then we show how to find it.

3.1. Existence of a trigonometric minimax approximation

We first introduce some definitions and theorems that are useful for our proof.

Definition 3.1 (Haar condition [12])**.** *Let $\mathcal{C}([\alpha,\beta])$ denote the set of all continuous functions on $[\alpha,\beta]$ and let $\{c_1,\dots,c_k\} \subset \mathcal{C}([\alpha,\beta])$. We say that $\{c_1,\dots,c_k\}$ satisfies the Haar condition on $[\alpha,\beta]$ if for any k distinct points $x_1,\dots,x_k \in [\alpha,\beta]$, the matrix*

$$\big(c_j(x_i)\big)_{i,j=1}^{k}$$

is invertible.

In particular, the trigonometric basis $\{c_1,\dots,c_k\} = \{1,\cos\eta x,\sin\eta x,\dots,\cos(d\eta x),\sin(d\eta x)\}$, where $k = 2d+1$, satisfies Haar's condition on $[\alpha,\beta]$ [13].

Theorem 3.1 (Heine–Borel theorem [39])**.** *A subset of $\mathbb{R}^k$ is compact if and only if it is closed and bounded.*

Theorem 3.2 (Closed preimage theorem [39])**.** *For a continuous function $g:\mathbb{R}^k\to\mathbb{R}$, if $\Phi\subset\mathbb{R}$ is closed, then its preimage*

$$g^{-1}(\Phi) = \{x\in\mathbb{R}^k : g(x)\in\Phi\}$$

2. Bian et al. [5] scale the plaintext space of f to $[-\delta\cdot\frac{q}{2},\delta\cdot\frac{q}{2}]$, with $\delta = 1/2$. Adopting the same δ, we set $[\alpha,\beta] = [-\frac{q}{4},\frac{q}{4}]$.

is closed in $\mathbb{R}^k$.

Theorem 3.3 (Weierstrass extreme value theorem [39])**.** *For a continuous function* $g : \Phi \to \mathbb{R}$*, where* Φ *is a nonempty compact subset of* $\mathbb{R}^k$*,* $\exists\ x_{\min}, x_{\max} \in \Phi$ *s.t.*

$$g(x_{\min}) \le g(x) \le g(x_{\max})\ \forall x \in \Phi.$$

Next, we prove the following lemma.

Lemma 3.1 (Mapping between $\mathcal{T}_d$ and $\mathbb{R}^k$)**.** *For any coefficient vector*

$$\theta = (c_0, a_1, b_1, \ldots, a_d, b_d) \in \mathbb{R}^k,$$

define

$$p_\theta(x) = c_0 + \sum_{m=1}^{d} \big(a_m \cos(m\eta x) + b_m \sin(m\eta x)\big).$$

Then, the map

$$\xi(\theta) = p_\theta$$

is a bijection, hence searching over $\mathcal{T}_d$ *is equivalent to searching over* $\mathbb{R}^k$*, i.e.,*

$$\min_{p\in\mathcal{T}_d} \|f_T - p\|_\infty = \min_{\theta\in\mathbb{R}^k} \|f_T - p_\theta\|_\infty.$$

Proof for this lemma is given in Appendix B.

Now, we are ready to prove the following theorem.

Theorem 3.4 (Existence of a trigonometric minimax approximation)**.** *For a target function*

$$f_T \in \mathcal{C}([\alpha, \beta]),$$

there exists a degree-d *trigonometric minimax approximation:*

$$\min_{p\in\mathcal{T}_d} \|f_T - p\|_\infty.$$

Proof. Define

$$\varepsilon(\theta) = \|f_T - p_\theta\|_\infty.$$

Then, we have

$$\varepsilon([0, \ldots, 0]) = \|f_T\|_\infty.$$

Define

$$\mathcal{A} = \left\{\theta \in \mathbb{R}^k : \varepsilon(\theta) \le \|f_T\|_\infty + 1\right\},$$

hence

$$\mathcal{A} = \varepsilon^{-1}\big([0, \|f_T\|_\infty + 1]\big).$$

Notice that $\mathcal{A}$ is nonempty as $[0, \ldots, 0] \in \mathcal{A}$. Since ε is continuous and $[0, \|f_T\|_\infty + 1]$ is closed in $\mathbb{R}$, by the closed preimage theorem (Theorem 3.2), $\mathcal{A}$ is closed in $\mathbb{R}^k$.

For any $\theta \in \mathcal{A}$, we have

$$\begin{aligned}\|p_\theta\|_\infty &\le \|f_T\|_\infty + \|f_T - p_\theta\|_\infty \\ &= \|f_T\|_\infty + \varepsilon(\theta) \\ &\le 2\|f_T\|_\infty + 1.\end{aligned}$$

Choose k distinct points

$$x_1, \ldots, x_k \in [\alpha, \beta].$$

Let

$$\{c_1, \ldots, c_k\} = \{1, \cos\eta x, \sin\eta x, \ldots, \cos(d\eta x), \sin(d\eta x)\},$$

and

$$\mathbf{Y} = \big(c_j(x_i)\big)_{i,j=1}^{k}.$$

Since $\{c_1, \ldots, c_k\}$ satisfies the Haar condition on $[\alpha, \beta]$, the matrix $\mathbf{Y}$ is invertible. Evaluating p_θ at $x_1, \ldots, x_k$ gives

$$\big(p_\theta(x_1), \ldots, p_\theta(x_k)\big)^\top = \mathbf{Y} \cdot \theta^\top.$$

Therefore,

$$\theta^\top = \mathbf{Y}^{-1}\big(p_\theta(x_1), \ldots, p_\theta(x_k)\big)^\top.$$

Since

$$|p_\theta(x_i)| \le \|p_\theta\|_\infty \le 2\|f_T\|_\infty + 1,$$

$\big(p_\theta(x_1), \ldots, p_\theta(x_k)\big)^\top$ is bounded. Then, $\theta^\top$ is bounded as $\mathbf{Y}^{-1}$ is fixed. Therefore, $\mathcal{A}$ is bounded.

As $\mathcal{A}$ is both closed and bounded, by the Heine–Borel theorem (Theorem 3.1), $\mathcal{A}$ is compact. Since ε is continuous on $\mathcal{A}$, by the Weierstrass extreme value theorem (Theorem 3.3), there exists $\theta^* \in \mathcal{A}$ s.t.

$$\varepsilon(\theta^*) \le \varepsilon(\theta),\ \forall\theta \in \mathcal{A}. \tag{1}$$

Equivalently,

$$\varepsilon(\theta^*) = \min_{\theta\in\mathcal{A}} \varepsilon(\theta).$$

Since $[0, \ldots, 0] \in \mathcal{A}$,

$$\varepsilon(\theta^*) \le \varepsilon([0, \ldots, 0]) = \|f_T\|_\infty.$$

By the definition of $\mathcal{A}$,

$$\varepsilon(\theta) > \|f_T\|_\infty + 1,\ \forall\theta \notin \mathcal{A}.$$

Therefore,

$$\varepsilon(\theta^*) < \varepsilon(\theta),\ \forall\theta \notin \mathcal{A}. \tag{2}$$

Combining Equations 1 and 2, we obtain

$$\varepsilon(\theta^*) = \min_{\theta\in\mathbb{R}^k} \varepsilon(\theta).$$

By Lemma 3.1, there exists $p^* \in \mathcal{T}_d$ s.t.

$$p^* = p_{\theta^*}.$$

Then,

$$\|f_T - p^*\|_\infty = \varepsilon(\theta^*) = \min_{\theta\in\mathbb{R}^k} \|f_T - p_\theta\|_\infty = \min_{p\in\mathcal{T}_d} \|f_T - p\|_\infty.$$

Therefore, the degree-d trigonometric minimax approximation of f_T exists. □

3.2. Finding a trigonometric minimax approximation

The following theorem plays a crucial role in finding a trigonometric minimax approximation.

Theorem 3.5 (Chebyshev alternation theorem [12])**.** *Suppose* $\{c_1, \ldots, c_k\} \subset \mathcal{C}([\alpha, \beta])$ *satisfies the Haar condition*

on $[\alpha, \beta]$. *Then, a function* $p \in \mathsf{span}\{c_1, \ldots, c_k\}$ *is a minimax approximation to* $f_T \in \mathcal{C}([\alpha, \beta])$ *on* $[\alpha, \beta]$ *if and only if there exist*

$$x_0 < x_1 < \cdots < x_k, \forall x_i \in [\alpha, \beta],$$

such that the error function

$$e(x) = f_T(x) - p(x)$$

satisfies

$$e(x_i) = -e(x_{i-1}) = \pm \max_{x \in [\alpha, \beta]} |e(x)|, \forall i = 1, \ldots, k,$$

which is called the alternation pattern. Such $\{x_i\}_{i=0}^k$ *are called reference nodes.*

Notice that $\mathsf{span}\{c_1, \ldots, c_k\} = \mathcal{T}_d$ when $\{c_1, \ldots, c_k\} = \{1, \cos \eta x, \sin \eta x, \ldots, \cos(d\eta x), \sin(d\eta x)\}$. Therefore, to find a trigonometric minimax approximation, we only need to find a polynomial $p \in \mathcal{T}_d$ that satisfies the alternation pattern, which can be achieved by the trigonometric Remez algorithm (described in Algorithm 1).

Building the target function. Bian et al. [5] observe that the approximation efficiency of Fourier series is closely related to the *smoothness* of the target function. To improve smoothness, after embedding f into each period of the mod function, they construct a bridge polynomial that connects the right endpoint (resp., left endpoint) of f in one period to the left endpoint (resp., right endpoint) in the adjacent period, thereby forming a smooth Fourier extension. In contrast, the Remez algorithm optimizes the approximation exclusively over the valid plaintext interval and does not attempt to fit the unused regions outside it. Consequently, our approach completely eliminates the need for bridge-polynomial construction.

Optimization for FHE. Directly running Algorithm 1 may result in large coefficients, which may amplify ciphertext noise during homomorphic evaluation. To address this, we replace Step 4 of Algorithm 1 with an inequality-constrained convex optimization:

$$\min_{\theta^{(i)}, E} \quad E + \lambda \|\theta^{(i)}\|_2^2 \quad \text{s.t.}$$

$$-E \le f_T(x_j^{(i-1)}) - p_{\theta^{(i)}}(x_j^{(i-1)}) \le E, \quad \forall j \in \{0, \ldots, k\}$$

where $\theta^{(i)}$ is the coefficient vector and

$$p_{\theta^{(i)}}(x) = c_0^{(i)} + \sum_{m=1}^{d} \left(a_m^{(i)} \cos(m\eta x) + b_m^{(i)} \sin(m\eta x) \right).$$

We further replace the stopping criterion in Step 8 with:

$$|M^{(i)} - M^{(i-1)}| < \epsilon,$$

where $M^{(i)} = \max_{x \in [\alpha, \beta]} |f_T(x) - p_{\theta^{(i)}}(x)|$.

Although this new step no longer computes the exact minimax approximation, it preserves the L_∞ optimization nature of the Remez algorithm while producing a trigonometric approximation with smaller coefficients. Thus, the resulting approximation trades a small amount of worst-case accuracy for improved stability in homomorphic evaluation.

4. Functional S2C

In AI models such as neural networks and transformers, linear and nonlinear layers typically alternate. Nonlinear layers can be naturally integrated into the EvalMod stage of bootstrapping, whereas linear layers must be evaluated separately via homomorphic matrix multiplication. In this section, we show how linear layers can instead be incorporated into the S2C transformation within bootstrapping.

Basic version. Suppose we wish to apply a known linear transformation $\mathbf{W} \in \mathbb{R}^{n \times n}$ to an SIMD encryption of $\mathbf{x} \in \mathbb{R}^{1 \times n}$:

$$\widetilde{\mathbf{y}} = \widetilde{\mathbf{x}} \cdot \mathbf{W}.$$

Recall that the S2C operation itself is a linear transformation:

$$\mathsf{S2C}(\widetilde{\mathbf{y}}) = \widetilde{\mathbf{y}} \cdot \mathbf{U}_0^\top,$$

where $\mathbf{U}_0^\top$ is the DFT matrix defined in Section 2.3. Since linear transformations are closed under composition, we have:

$$\mathsf{S2C}(\widetilde{\mathbf{x}} \cdot \mathbf{W}) = \widetilde{\mathbf{x}} \cdot (\mathbf{W} \cdot \mathbf{U}_0^\top),$$

where $\mathbf{W} \cdot \mathbf{U}_0^\top$ can be precomputed offline. Therefore, we define a *functional S2C* operation:

$$\mathsf{S2C}_{\mathbf{W}}(\widetilde{\mathbf{x}}) = \mathsf{S2C}(\widetilde{\mathbf{x}} \cdot \mathbf{W}) = \widetilde{\mathbf{x}} \cdot (\mathbf{W} \cdot \mathbf{U}_0^\top).$$

Some linear layers appearing in transformer models also include an affine term:

$$\widetilde{\mathbf{y}} = \widetilde{\mathbf{x}} \cdot \mathbf{W} + \mathbf{b}.$$

Similarly, for an affine transformation,

$$\begin{aligned} \mathsf{S2C}(\widetilde{\mathbf{x}} \cdot \mathbf{W} + \mathbf{b}) &= (\widetilde{\mathbf{x}} \cdot \mathbf{W} + \mathbf{b}) \cdot \mathbf{U}_0^\top \\ &= \widetilde{\mathbf{x}} \cdot (\mathbf{W} \cdot \mathbf{U}_0^\top) + \mathbf{b} \cdot \mathbf{U}_0^\top, \end{aligned}$$

where both $\mathbf{W} \cdot \mathbf{U}_0^\top$ and $\mathbf{b} \cdot \mathbf{U}_0^\top$ can be precomputed offline. We therefore define

$$\mathsf{S2C}_{\mathbf{W},\mathbf{b}}(\widetilde{\mathbf{x}}) = \mathsf{S2C}(\widetilde{\mathbf{x}} \cdot \mathbf{W} + \mathbf{b}).$$

Decomposition of DFT matrix. The original DFT matrix $\mathbf{U}_0^\top$ admits an FFT-style recursive factorization into $\log n$ sparse matrices:

$$\mathbf{U}_0^\top = \mathbf{M}_1 \cdots \mathbf{M}_{\log n},$$

where each factor $\mathbf{M}_i$ contains only three nonzero cyclic diagonals: one main diagonal and two symmetric off-diagonals. Consequently, each sparse factor can be applied to $\widetilde{\mathbf{x}}$ using three ciphertext-plaintext multiplications and two rotations. The resulting S2C transformation requires a total of $3 \log n$ ciphertext-plaintext multiplications and $2 \log n$ rotations, but consumes $\log n$ multiplicative levels. To reduce level consumption, Chen et al. [9] propose a level-collapsing strategy that trades computation for fewer levels. The idea is to merge several adjacent factors into a single denser one.

Algorithm 1 The trigonometric Remez algorithm

Input: The target function $f_T(x)$, interval $[\alpha, \beta]$, trigonometric degree d, tolerance ϵ, maximum iterations $K_{\max}$
Output: A trigonometric polynomial $p(x)$ and reference nodes $\{x_j\}_{j=0}^{k}$

Set $k = 2d + 1$
Initialize reference nodes $\{x_j^{(0)}\}_{j=0}^{k}$ with $k+1$ equispaced nodes on $[\alpha, \beta]$
for $i \leftarrow 1$ to $K_{\max}$ **do**
    Solve the linear system

$$\left\{ c_0^{(i)} + \sum_{m=1}^{d} \left(a_m^{(i)} \cos\big(m\eta x_j^{(i-1)}\big) + b_m^{(i)} \sin\big(m\eta x_j^{(i-1)}\big) \right) + (-1)^j \rho^{(i)} = f_T\big(x_j^{(i-1)}\big) \right\}$$
$$j = 0, 1, \ldots, k$$

    to obtain $\{c_0^{(i)}, a_1^{(i)}, b_1^{(i)}, \ldots, a_d^{(i)}, b_d^{(i)}\}$ and $\rho^{(i)}$
    Define $p^{(i)}(x) = c_0^{(i)} + \sum_{m=1}^{d} \left(a_m^{(i)} \cos\big(m\eta x\big) + b_m^{(i)} \sin\big(m\eta x\big) \right)$ and $e^{(i)}(x) = f_T(x) - p^{(i)}(x)$
    Locate local extrema of $e^{(i)}(x)$ on $[\alpha, \beta]$
    Select $k+1$ extremum candidates $\{x'_j\}_{j=0}^{k}$ such that $e^{(i)}(x'_j)$ alternates in sign
    if $i > 1$ and $\left|\rho^{(i)} - \rho^{(i-1)}\right| < \epsilon$ **then**
        break
    end if
    Update reference nodes: $\{x_j^{(i)}\}_{j=0}^{k} \leftarrow \{x'_j\}_{j=0}^{k}$
end for
return $p^{(i)}(x)$ and $\{x_j^{(i)}\}_{j=0}^{k}$

In practice, the $\log n$ sparse factors are collapsed into two blocks, reducing the level consumption from $\log n$ to two.

In our functional S2C, the fused matrix $\mathbf{W} \cdot \mathbf{U}_0^\top$ no longer preserves the sparse FFT structure. Fortunately, the interleaved row-packing format enables us to exploit both the functional S2C and the decomposition of $\mathbf{U}_0^\top$. Concretely, let $B = n/n'$ and

$$\widetilde{\mathbf{x}} = \mathsf{ILPack}(\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(B-1)});$$

given $\mathbf{W} \in \mathbb{R}^{n' \times n'}$, the "*row-packing* $\times$ *plaintext*" routine (cf. Section 2.5) computes:

$$\begin{aligned} \widetilde{\mathbf{x}} \cdot \tau(\mathbf{W}) &= \widetilde{\mathbf{x}} \cdot (\mathbf{W} \otimes \mathbf{I}_B) \\ &= \mathsf{ILPack}(\mathbf{x}^{(0)} \cdot \mathbf{W}, \ldots, \mathbf{x}^{(B-1)} \cdot \mathbf{W}), \end{aligned}$$

where $\otimes$ denotes the Kronecker product and $\mathbf{I}_B$ is the $B \times B$ identity matrix.

We collapse the $\log n$ factors of $\mathbf{U}_0^\top$ into the following two blocks:

$$\mathbf{U}_0^\top = \underbrace{\mathbf{M}_1 \cdots \mathbf{M}_{\log B}}_{\mathbf{S}_0} \underbrace{\mathbf{M}_{\log B + 1} \cdots \mathbf{M}_{\log n}}_{\mathbf{S}_1} = \mathbf{S}_0 \cdot \mathbf{S}_1.$$

As shown in Section 3.1 of [29], the first block $\mathbf{S}_0$ can be expressed as

$$\mathbf{S}_0 = \mathbf{I}_{n'} \otimes \mathbf{L}_B, \text{ where } \mathbf{L}_B \in \mathbb{R}^{B \times B},$$

which has nonzero diagonals only at offsets

$$-(B-1), \ \ldots, \ -1, \ 0, \ 1, \ \ldots, \ B-1.$$

Meanwhile, the second block $\mathbf{S}_1$ has nonzero diagonals only at offsets

$$0, \ B, \ 2B, \ \ldots, \ (n'-1)B.$$

Our key observation is that $\tau(\mathbf{W})$ commutes with $\mathbf{S}_0$ by the *mixed-product property* of the Kronecker product. Specifically,

$$\begin{aligned} \tau(\mathbf{W}) \cdot \mathbf{S}_0 &= (\mathbf{W} \otimes \mathbf{I}_B) \cdot (\mathbf{I}_{n'} \otimes \mathbf{L}_B) \\ &= (\mathbf{W} \cdot \mathbf{I}_{n'}) \otimes (\mathbf{I}_B \cdot \mathbf{L}_B) \\ &= \mathbf{W} \otimes \mathbf{L}_B \end{aligned}$$

and

$$\begin{aligned} \mathbf{S}_0 \cdot \tau(\mathbf{W}) &= (\mathbf{I}_{n'} \otimes \mathbf{L}_B) \cdot (\mathbf{W} \otimes \mathbf{I}_B) \\ &= (\mathbf{I}_{n'} \cdot \mathbf{W}) \otimes (\mathbf{L}_B \cdot \mathbf{I}_B) \\ &= \mathbf{W} \otimes \mathbf{L}_B \end{aligned}$$

hence,

$$\tau(\mathbf{W}) \cdot \mathbf{S}_0 = \mathbf{S}_0 \cdot \tau(\mathbf{W}).$$

This allows us to rewrite the fused transformation as:

$$\begin{aligned} \widetilde{\mathbf{x}} \cdot \tau(\mathbf{W}) \cdot \mathbf{U}_0^\top &= \widetilde{\mathbf{x}} \cdot \tau(\mathbf{W}) \cdot \big(\mathbf{S}_0 \cdot \mathbf{S}_1\big) \\ &= \widetilde{\mathbf{x}} \cdot \mathbf{S}_0 \cdot \big(\tau(\mathbf{W}) \cdot \mathbf{S}_1\big). \end{aligned}$$

Importantly, fusing $\tau(\mathbf{W})$ with $\mathbf{S}_1$ preserves the sparse diagonal structure, since both matrices have nonzero diagonals only at offsets

$$0, \ B, \ 2B, \ \ldots, \ (n'-1)B.$$

Consequently, our functional S2C requires $O(n' + B)$ ciphertext-plaintext multiplications and $O(\sqrt{n'} + \sqrt{B})$ rotations, while consuming 2 levels. Notably, this complexity is identical to that of the original S2C transformation:

$$\mathsf{S2C}(\widetilde{\mathbf{x}}) = \widetilde{\mathbf{x}} \cdot \mathbf{U}_0^\top = \widetilde{\mathbf{x}} \cdot \mathbf{S}_0 \cdot \mathbf{S}_1,$$

implying that our functional S2C comes essentially for free.

Functional C2S. Since both S2C and C2S are linear transformations, one may similarly fuse a linear transformation into C2S through composition. However, C2S is performed after the ModRaise step, where ciphertexts have already been lifted from modulus q_0 to a much larger modulus Q. Consequently, all homomorphic operations fused into C2S must be carried out under this enlarged modulus. Since the cost of homomorphic operations such as ciphertext multiplications, rotations, and key switching grows with the modulus size, performing the fused transformation after ModRaise would substantially increase both computation and memory overhead. In contrast, S2C is executed before ModRaise, where the ciphertext still operates under the smaller modulus q_0. Therefore, incorporating linear transformations into S2C allows the fused computation to be performed at a significantly lower cost, making functional S2C a more practical design choice.

5. The NISTI Framework

In this section, we show how we use our improved FBS for NISTI.

5.1. Homomorphic packing conversion

Looking ahead, we require homomorphic conversions between $\widetilde{\mathbf{X}}^{\mathsf{col}}$ and $\widetilde{\mathbf{X}}^{\mathsf{row}}$ in both directions. Suppose $\mathbf{X} \in \mathbb{R}^{n\times n}$ is a square matrix and we wish to convert $\widetilde{\mathbf{X}}^{\mathsf{col}}$ into $\widetilde{\mathbf{X}}^{\mathsf{row}}$. A straightforward approach extracts each entry from its corresponding column ciphertext and rotates it directly to its target position in the output row ciphertext. Specifically, $\forall j \in [n]$, it computes

$$\widetilde{\mathbf{X}}^{\mathsf{row}}_j = \sum_{i=0}^{n-1} \mathsf{Rot}_{(j-i)}(\mathbf{m}_j \cdot \widetilde{\mathbf{X}}^{\mathsf{col}}_i),$$

where $\mathbf{m}_j \in \{0,1\}^n$ is the j-th one-hot vector. This approach requires n^2 ciphertext-plaintext multiplications and n^2 rotations, consuming one level. A natural optimization is to employ a butterfly-style transpose network, reducing the cost to $O(n \log n)$ ciphertext-plaintext multiplications and $O(n \log n)$ rotations, but at the cost of $\log n$ levels.

We propose a novel optimization. Using $\mathsf{Rot}_{j-i} = \mathsf{Rot}_j(\mathsf{Rot}_{-i})$, we obtain

$$\begin{aligned}\widetilde{\mathbf{X}}^{\mathsf{row}}_j &= \mathsf{Rot}_j\left(\sum_{i=0}^{n-1} \mathsf{Rot}_{-i}\big(\mathbf{m}_j \cdot \widetilde{\mathbf{X}}^{\mathsf{col}}_i\big)\right) \\ &= \mathsf{Rot}_j\left(\sum_{i=0}^{n-1} \mathbf{m}_{i+j} \cdot \mathsf{Rot}_{-i}\big(\widetilde{\mathbf{X}}^{\mathsf{col}}_i\big)\right),\end{aligned}$$

where the term $\mathsf{Rot}_{-i}\big(\widetilde{\mathbf{X}}^{\mathsf{col}}_i\big)$ $\forall i \in [n]$ can be precomputed and reused across all j. As a result, the conversion requires n^2 ciphertext-plaintext multiplications but only $2n$ rotations, while still consuming a single level.

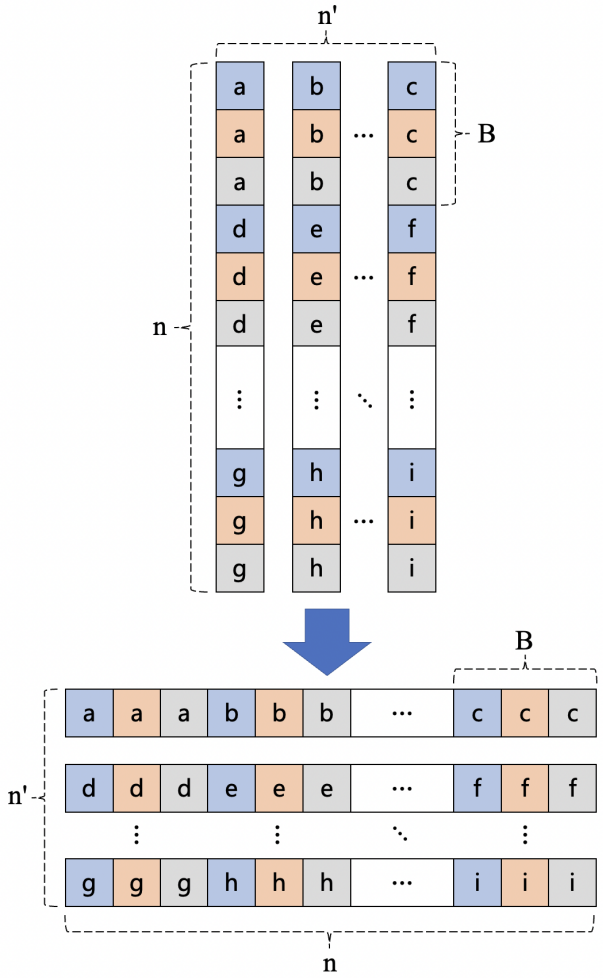


Figure 4: Interleaved packing conversion.

In our target setting, $\mathbf{X} \in \mathbb{R}^{n\times m}$ is encrypted such that each ciphertext column contains an interleaved packing of $B = n/n'$ vectors, where $n'|m$. We partition $\widetilde{\mathbf{X}}^{\mathsf{col}}$ into $B' = m/n'$ submatrices, each of dimension $n \times n'$. For each submatrix, we compute

$$\forall j \in [n'],\ \widetilde{\mathbf{X}}^{\mathsf{row}}_j = \mathsf{Rot}_{j\cdot B}\left(\sum_{i=0}^{n'-1} \mathbf{m}'_{i+j} \cdot \mathsf{Rot}_{-i\cdot B}\big(\widetilde{\mathbf{X}}^{\mathsf{col}}_i\big)\right),$$

where $\mathbf{m}'_{i+j} \in \{0,1\}^n$ satisfies $\mathbf{m}'_{i+j}[k] = 1$ if $\lfloor k/B \rfloor = i + j$. Figure 4 visualizes the conversion for a single such submatrix. This transformation produces B' row-packed matrices, each of dimension $n' \times n$. The whole process requires $n'm$ ciphertext-plaintext multiplications and only $2m$ rotations (recall that interleaved packing can perform B rotations in a batch with a single homomorphic rotation), while consuming one level.

The reverse conversion, from $\widetilde{\mathbf{X}}^{\mathsf{row}}$ to $\widetilde{\mathbf{X}}^{\mathsf{col}}$, can be performed analogously and achieves the same asymptotic complexity.

1) $\mathbf{Q}_h||\mathbf{K}_h||\mathbf{V}_h = \mathbf{X}\cdot(\mathbf{W}_{Q,h}||\mathbf{W}_{K,h}||\mathbf{W}_{V,h}) \in \mathbb{R}^{n\times d}||\mathbb{R}^{n\times d}||\mathbb{R}^{n\times d}$:
   - Take as input $\widetilde{\mathbf{X}}^{\mathsf{col}}$ (where $\mathbf{X}\in\mathbb{R}^{n\times m}$), representing an interleaved column-packing of $B = n/n'$ matrices, each of dimension $n'\times m$.
   - Given $\mathbf{W}_{Q,h}\in\mathbb{R}^{m\times d}$, compute $\widetilde{\mathbf{Q}}_h^{\mathsf{col}} = \widetilde{\mathbf{X}}^{\mathsf{col}}\cdot\mathbf{W}_{Q,h}$.
   - $\widetilde{\mathbf{K}}_h^{\mathsf{col}}$ and $\widetilde{\mathbf{V}}_h^{\mathsf{col}}$ can be computed in the same way.
2) $\mathbf{A}_h = \mathbf{Q}_h\cdot\mathbf{K}_h^\top \in \mathbb{R}^{n\times n}$:
   - Let $\mathbf{F}_h = \mathbf{K}_h^\top$, then $\widetilde{\mathbf{F}}_h^{\mathsf{row}} = \widetilde{\mathbf{K}}_h^{\mathsf{col}}$, compute $\widetilde{\mathbf{A}}_h^{\mathsf{diag}} = \widetilde{\mathbf{Q}}_h^{\mathsf{col}}\cdot\widetilde{\mathbf{F}}_h^{\mathsf{row}}$.
3) $\mathbf{B}_h = \mathsf{Softmax}\Big(\frac{\mathbf{A}_h}{\sqrt{d}}\Big) \in \mathbb{R}^{n\times n}$:
   - Define $\mathbf{G}_h\in\mathbb{R}^{n\times n}$ with each of its entries $g = \exp(a/\sqrt{d})$, where a is the corresponding entry in $\mathbf{A}_h$.
   - $\forall i\in[n]$, compute the i-th diagonal of $\mathbf{G}_h$: $\widetilde{\mathbf{G}}_h^{\mathsf{diag}}[i] = \mathsf{EvalMod}_{\mathsf{exp}}\circ\mathsf{C2S}\circ\mathsf{ModRaise}\circ\mathsf{S2C}(\widetilde{\mathbf{A}}_h^{\mathsf{diag}}[i]\cdot\frac{1}{\sqrt{d}})$.
   - Compute $\widetilde{\mathbf{g}}_h = \sum_{i=0}^{n-1}\widetilde{\mathbf{G}}_h^{\mathsf{diag}}[i]$ and $\widetilde{\mathbf{g}}_h^{\mathsf{inv}} = \mathsf{EvalMod}_{\mathsf{inv}}\circ\mathsf{C2S}\circ\mathsf{ModRaise}\circ\mathsf{S2C}(\widetilde{\mathbf{g}}_h)$.
   - $\forall i\in[n]$, compute $\widetilde{\mathbf{B}}_h[i] = \widetilde{\mathbf{G}}_h^{\mathsf{diag}}[i]\cdot\widetilde{\mathbf{g}}_h^{\mathsf{inv}}$.
4) $\mathbf{C}_h = \mathbf{B}_h\cdot\mathbf{V}_h\in\mathbb{R}^{n\times d}$:
   - Compute $\widetilde{\mathbf{C}}_h^{\mathsf{col}} = \widetilde{\mathbf{B}}_h^{\mathsf{diag}}\cdot\widetilde{\mathbf{V}}_h^{\mathsf{col}}$.
5) $\mathbf{D} = \alpha\cdot\tanh\big(\beta\cdot(\mathbf{C}_0||\ldots||\mathbf{C}_{H-1})\cdot\mathbf{W}_0\big)+\gamma\in\mathbb{R}^{n\times m}$:
   a) Let $\mathbf{H} = \beta\cdot\mathbf{C}_0||\ldots||\mathbf{C}_{H-1}\in\mathbb{R}^{n\times(d\cdot H)}$, then $\widetilde{\mathbf{H}}^{\mathsf{col}} = \beta\cdot\widetilde{\mathbf{C}}_0^{\mathsf{col}}||\ldots||\widetilde{\mathbf{C}}_{H-1}^{\mathsf{col}}$.
   b) Convert $\widetilde{\mathbf{H}}^{\mathsf{col}}$ into $\widetilde{\mathbf{H}}^{\mathsf{row}}$, which requires $n'\cdot(d\cdot H)$ ciphertext-plaintext multiplications and $2\cdot d\cdot H$ rotations.
   c) Compute $\widetilde{\mathbf{D}}^{\mathsf{row}} = \alpha\cdot\mathsf{EvalMod}_{\mathsf{tanh}}\circ\mathsf{C2S}\circ\mathsf{ModRaise}\circ\mathsf{S2C}_{\mathbf{W}_0}(\widetilde{\mathbf{H}}^{\mathsf{row}})+\gamma$, where $\mathsf{S2C}_{\mathbf{W}_0}(\widetilde{\mathbf{H}}^{\mathsf{row}})$ requires $n\times m$ ciphertext-plaintext multiplications and $2n\cdot\sqrt{m}$ rotations.
6) $\mathbf{E} = \mathsf{GELU}\big(\mathbf{D}\cdot\mathbf{W}_1+\mathbf{b}_1\big)\in\mathbb{R}^{n\times k}$:
   - Given $\mathbf{W}_1\in\mathbb{R}^{m\times k}$ and $\mathbf{b}_1\in\mathbb{R}^{n\times k}$, compute $\widetilde{\mathbf{E}}^{\mathsf{row}} = \mathsf{EvalMod}_{\mathsf{GELU}}\circ\mathsf{C2S}\circ\mathsf{ModRaise}\circ\mathsf{S2C}_{\mathbf{W}_1,\mathbf{b}_1}(\widetilde{\mathbf{D}}^{\mathsf{row}})$.
7) $\mathbf{X} = \alpha\cdot\tanh\big(\beta\cdot(\mathbf{E}\cdot\mathbf{W}_2+\mathbf{b}_2)\big)+\gamma\in\mathbb{R}^{n\times m}$:
   - Given $\mathbf{W}_2\in\mathbb{R}^{k\times m}$, $\mathbf{b}_2\in\mathbb{R}^{n\times m}$, compute $\widetilde{\mathbf{X}}^{\mathsf{row}} = \alpha\mathsf{EvalMod}_{\mathsf{tanh}}\circ\mathsf{C2S}\circ\mathsf{ModRaise}\circ\mathsf{S2C}_{\beta\mathbf{W}_2,\beta\mathbf{b}_2}(\widetilde{\mathbf{E}}^{\mathsf{row}})+\gamma$.
   - Convert $\widetilde{\mathbf{X}}^{\mathsf{row}}$ into $\widetilde{\mathbf{X}}^{\mathsf{col}}$ and input it into the next iteration.

Figure 3: The basic NISTI protocol.

5.2. The basic protocol

Figure 3 shows the basic protocol (the steps are consistent with the transformer described in Section 2.2). The original input $\widetilde{\mathbf{X}}^{\mathsf{col}}$ is an interleaved column-packing of $B = n/n'$ input matrices, each of dimension $n'\times m$. In Step 5.b, $\widetilde{\mathbf{H}}^{\mathsf{row}}$ consists of $(d\cdot H)/n'$ row-packed matrices, each of dimension $n'\times n$ (cf. Section 5.1). Following the interleaved "*row-packing* $\times$ *plaintext*" routine (cf. Section 2.5), $\mathbf{W}_0$ needs to be expanded into $\tau(\mathbf{W})\in\mathbb{R}^{n\times n}$, which can then be pre-fused with $\mathbf{U}_0$. The same treatment applies to Steps 6 and 7.

5.3. Batch S2C

We could significantly reduce the overhead of Step 5 (in Figure 3) based on the fact that $\widetilde{\mathbf{H}}$ is originally in column-packing. Recall that

$$\mathsf{S2C}_{\mathbf{W}_0}(\widetilde{\mathbf{H}}^{\mathsf{row}}) = \mathsf{S2C}(\widetilde{\mathbf{H}}^{\mathsf{row}}\cdot\mathbf{W}_0) = \widetilde{\mathbf{H}}^{\mathsf{row}}\cdot\mathbf{W}_0\cdot\mathbf{U}_0^\top.$$

Let

$$\widetilde{\mathbf{L}}^{\mathsf{row}} = \widetilde{\mathbf{H}}^{\mathsf{row}}\cdot(\mathbf{W}_0\cdot\mathbf{U}_0^\top).$$

Then,

$$\widetilde{\mathbf{D}}^{\mathsf{row}} = \alpha\cdot\mathsf{EvalMod}_{\mathsf{tanh}}\circ\mathsf{C2S}\circ\mathsf{ModRaise}(\widetilde{\mathbf{L}}^{\mathsf{row}})+\gamma.$$

To this end, we could compute $\widetilde{\mathbf{L}}^{\mathsf{row}}$ as follows:

1) Compute $\widetilde{\mathbf{L}}^{\mathsf{col}}$:

$$\forall i \in [m],\ \widetilde{\mathbf{L}}^{\mathsf{col}}[i] = \sum_{j=1}^{d\cdot H} \widetilde{\mathbf{H}}^{\mathsf{col}}[j] \cdot (\mathbf{W}_0 \cdot \mathbf{U}_0^{\top})[j,i],$$

where $\mathbf{U}_0 \in \mathbb{C}^{m\times m}$ is as follows (assuming $m|N$ and let $k = N/m$):

$$\mathbf{U}_0 = \begin{bmatrix} 1 & \zeta_0^k & \zeta_0^{2k} & \cdots & \zeta_0^{(m-1)k} \\ 1 & \zeta_1^k & \zeta_1^{2k} & \cdots & \zeta_1^{(m-1)k} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \zeta_{m-1}^k & \zeta_{m-1}^{2k} & \cdots & \zeta_{m-1}^{(m-1)k} \end{bmatrix} \in \mathbb{C}^{m\times m}.$$

This step requires $m \times (d \cdot H)$ ciphertext-plaintext multiplications.

2) Turn $\widetilde{\mathbf{L}}^{\mathsf{col}}$ into $\widetilde{\mathbf{L}}^{\mathsf{row}}$, which requires $n \cdot m$ ciphertext-plaintext multiplications and $n + m$ rotations.

In practice $m = d \cdot H$, hence we save $O(n \cdot \sqrt{m})$ rotations.

5.4. Further optimizations

Lazy key switching. The "*column-packing* $\times$ *row-packing*" routine computes:

$$\widetilde{\mathbf{Z}}^{\mathsf{diag}}[i] = \sum_{j=0}^{d-1} \widetilde{\mathbf{X}}^{\mathsf{col}}[j] \cdot \mathsf{Rot}_i(\widetilde{\mathbf{Y}}^{\mathsf{row}}[j])\ \forall i \in [n],$$

which requires nd rotations, hence nd key switching operations. We use *lazy key switching* [8] to reduce the number of key switching operations from nd to $2d$.

Hoisting. The "*diagonal-packing* $\times$ *column-packing*" routine computes:

$$\widetilde{\mathbf{Z}}^{\mathsf{col}}[i] = \sum_{j=0}^{n-1} \widetilde{\mathbf{X}}^{\mathsf{diag}}[j] \cdot \mathsf{Rot}_j(\widetilde{\mathbf{Y}}^{\mathsf{col}}[i])\ \forall i \in [d],$$

which requires n rotations for each $\widetilde{\mathbf{Y}}^{\mathsf{col}}[i]$. We use *hoisting* [24] to precompute the input-dependent operations such as ciphertext decomposition and reuse it across all rotations.

Lazy relinearization and lazy rescaling. We further optimize the performance of matrix multiplications using *Lazy relinearization* and *lazy rescaling* [7], [10], [6], [30], which delay the relinearization and rescaling steps until after accumulating ciphertext products into a sum, rather than applying them immediately after each ciphertext multiplication.

6. Evaluation

We implement our scheme in C++, using OpenFHE v1.5.1 for CKKS operations and FBS, while employing OpenMP for multi-threaded parallelization. We set the polynomial degree to $N = 2^{16}$, providing $n = 2^{15}$ slots per ciphertext. To achieve 128-bit security, we set the maximum CKKS modulus $\lceil \log_2(QP) \rceil \approx 1707$ bits, where P is the auxiliary modulus for key-switching. We configure the scheme with a maximum level budget of 24, with FBS consuming 18 levels. Our CKKS setup employs a `SPARSE_TERNARY` secret key with Hamming weight 192, the `FLEXIBLEAUTO` scaling technique, and hybrid BV–GHS key-switching. The scaling factor is $\Delta \approx 2^{49}$.

6.1. Evaluation of FBS

We evaluate the precision and latency of FBS across a range of functions, including exp, GELU, Sigmoid, and mod, and compare against the state-of-the-art FBS scheme of Bian et al. [5]. Following Bian et al., we measure precision as $\log \epsilon^{-1}$, where ϵ denotes the output error.

As reported in Table 2, our approach achieves a precision improvement of 4.21–14.36 bits over Bian et al. while using the same approximation degree, and hence incurring essentially the same latency. In particular, with approximation degrees of only 13 for exp and 34 for GELU, our method attains more than 33 bits of precision for both functions, thereby providing a solid foundation for NISTI.

6.2. Evaluation of the NISTI framework

We primarily compare our framework against MOAI [43], the state-of-the-art NISTI framework. All experiments are conducted on an Intel(R) Xeon(R) Platinum 8358 CPU running at 2.60GHz with 64 cores. Our evaluation setting closely follows that of MOAI, and the reproduced results are consistent with those reported in the original paper.

The evaluated model is a BERT-base architecture comprising 12 transformer layers, 12 attention heads, a hidden dimension of 768, and a feed-forward dimension of 3072. The input sequence length is fixed at 128 tokens, and each query is processed with a batch size of 256 sequences. For our framework, we replace LayerNorm with Dynamic Tanh (DyT). We distill BERT-DyT from the original BERT, treating the latter as the teacher model.

Accuracy. We evaluate inference accuracy on three datasets from the GLUE benchmark [40] – RTE, SST-2, and MRPC – which is widely used for evaluating transformer models. Potential accuracy degradation may arise from two sources: the replacement of LayerNorm with DyT and the approximation errors introduced by our framework. As shown in Table 3, BERT-DyT matches the accuracy of the original BERT model. Moreover, our framework introduces virtually no additional accuracy loss, achieving nearly identical accuracy to BERT-DyT across all three datasets.

Table 3: Accuracy evaluation.

Dataset	#Test	Unencrypted		Encrypted
		BERT	BERT-DyT	
RTE	277	71.12	71.12	69.39
SST-2	872	93.00	93.00	92.43
MRPC	408	86.76	86.76	85.78

Table 2: Comparison of FBS with Bian et al. [5] over different functions.

Function	Range	Scheme	Parameters		Precision (bits)	Latency (s)	Amortized time (ms)
			degree	smoothness			
exp	$[-2, 2]$	Bian et al.	13	12	19.17	27.56	0.8411
		Ours	13	–	33.53	26.51	0.8089
Sigmoid	$[-8, 8]$	Bian et al.	26	9	25.52	26.93	0.8217
		Ours	26	–	31.15	26.47	0.8076
GELU	$[-8, 8]$	Bian et al.	34	8	28.99	27.29	0.8328
		Ours	34	–	33.20	26.40	0.8052
mod	$[-0.5, 0.5]$	Bian et al.	13	10	21.83	26.63	0.8126
		Ours	13	–	33.76	27.42	0.8369

End-to-end performance. Figure 5 shows the end-to-end performance amortized over 256 inputs. Our framework takes 349.5s to complete the entire inference, which is 1.9× faster than MOAI. In terms of communication, our framework only consumes 16.1MB of bandwidth, which is a 3× reduction over MOAI.

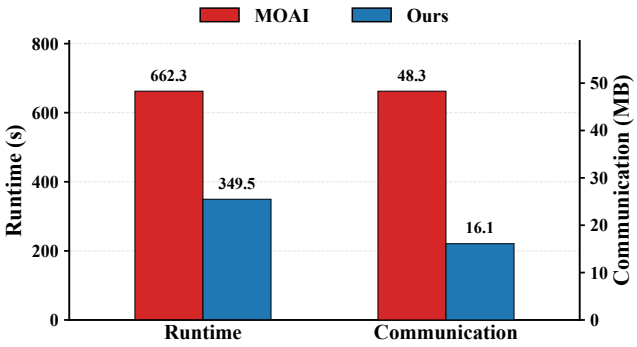


Figure 5: End-to-end performance.

Table 4 lists the runtime of each individual step in our framework, comparing it with MOAI. For the first two linear layers, we use exactly the same algorithms as MOAI; thanks to our smaller ciphertext size, we achieve upto 55.8× speedup. For Softmax, our FBS approach is also faster than MOAI's polynomial-based method, once again due to the reduced ciphertext size. MOAI performs bootstrapping during the two normalization layers, making it significantly slower than our approach. The only operation that is slower in our framework is $\mathbf{E} = \mathsf{GELU}(\mathbf{D} \cdot \mathbf{W}_1 + \mathbf{b}_1)$. This is because the dimension at this stage is large, and previous works typically avoid bootstrapping at this stage. One natural question is whether it is possible to compute GELU using the traditional approach instead of FBS. Doing so would require enlarging the ciphertext to support more levels during the preceding FBS step, which would increase the total runtime.

7. Related Work

FBS. FHEW [20] and TFHE [18] pioneered the concept of FBS by embedding a lookup table (LUT) into the bootstrapping accumulator, enabling the evaluation of arbitrary functions on a single encrypted bit or small integer at the cost of a bootstrapping operation. Their per-ciphertext latency and exponential scaling with precision remained a bottleneck. To leverage SIMD amortization, subsequent works adapted FBS to the BFV/BGV schemes [34], [28], constructing LUT-specific polynomials evaluated during bootstrapping. Early CKKS bootstrapping approximated the modular reduction (identity) using a scaled sine function [15], but extending this to arbitrary functions required new interpolation techniques. Notably, Bae et al. [4] proposed a trigonometric Lagrange interpolation on roots of unity, while Alexandru [2] introduced trigonometric Hermite interpolation. Despite these advances, precision remained limited (typically ≤ 12 bits) due to the inherent discretization of LUTs and the slow convergence of direct trigonometric approximations. Diverging from the interpolation paradigm, Bian et al. [5] construct a highly smooth Fourier extension of the target function by adding a bridge polynomial, improving precision by over 10 bits while reducing latency.

Remez for bootstrapping. Lee et al. [31] extend the classical Remez algorithm to handle multi-interval domains: a union of many disjoint closed intervals, as required for the modular reduction function in CKKS bootstrapping. They propose a multi-interval Remez algorithm with a maximum-absolute-sum criterion for selecting reference points, proven to converge, and they combine it with an arcsin-composition to improve precision. RBOOT [41] employs the classical Remez algorithm in the polynomial basis (specifically Chebyshev) to approximate arcsin, which is then composed with sine/cosine to evaluate ReLU during bootstrapping. It is unclear how such an approach can be easily generalized to arbitrary functions.

Fusing linear operations with bootstrapping. NeuJeans [29] introduces a Coefficients-in-Slot (CinS) encoding

Table 4: Runtime breakdown amortized over 256 inputs. MOAI still uses LayerNorm instead of Dynamic Tanh.

Operation	Dimension	Ours		MOAI [43]	
		Level	Time (s)	Level	Time (s)
$\mathbf{Q}\|\|\mathbf{K}\|\|\mathbf{V} = \mathbf{X} \cdot (\mathbf{W}_Q\|\|\mathbf{W}_K\|\|\mathbf{W}_V)$	$(\mathbb{R}^{128\times768} \times \mathbb{R}^{768\times768}) \times 3$	$5 \to 4$	13.1	$15 \to 14$	36.3
$\mathbf{A}_h = \mathbf{Q}_h \cdot \mathbf{K}_h^\top$	$(\mathbb{R}^{128\times64} \times \mathbb{R}^{64\times128}) \times 12$	$4 \to 3$	1.22	$14 \to 13$	68.1
$\mathbf{B}_h = \mathsf{Softmax}\left(\frac{\mathbf{A}_h}{\sqrt{d}}\right)$	$(\mathbb{R}^{128\times128}) \times 12$	$3 \to 24 \to 4$	71.2	$13 \to 3$	92.3
$\mathbf{C}_h = \mathbf{B}_h \cdot \mathbf{V}_h$	$(\mathbb{R}^{128\times128} \times \mathbb{R}^{128\times64}) \times 12$	$4 \to 3$	1.91	$3 \to 2$	1.63
$\mathbf{D} = \alpha \cdot \tanh\big(\beta \cdot \mathbf{C} \cdot \mathbf{W}_0\big) + \gamma$	$\mathbb{R}^{128\times768} \times \mathbb{R}^{768\times768}$	$3 \to 21 \to 3$	44.6	$2 \to 35 \to 10$	203.2
$\mathbf{E} = \mathsf{GELU}\big(\mathbf{D} \cdot \mathbf{W}_1 + \mathbf{b}_1\big)$	$\mathbb{R}^{128\times768} \times \mathbb{R}^{768\times3072}$	$3 \to 21 \to 3$	141.1	$10 \to 2$	43.4
$\mathbf{X} = \alpha \cdot \tanh\big(\beta \cdot (\mathbf{E} \cdot \mathbf{W}_2 + \mathbf{b}_2)\big) + \gamma$	$\mathbb{R}^{128\times3072} \times \mathbb{R}^{3072\times768}$	$3 \to 24 \to 5$	76.4	$2 \to 35 \to 15$	217.4
Total	–	–	**349.5**	–	**662.3**

that enables convolution operations to be evaluated without costly slot rotations. The key observation is that CinS encoding can be obtained by executing the initial stages of the DFT transformation, which overlap with part of the S2C transformation. By extracting these shared stages from S2C and reusing them to construct the CinS representation, NeuJeans reduces the overall computational cost of CNN inference. However, this optimization is tailored specifically to convolution operations and does not naturally support arbitrary linear or affine transformations. MaMBo [3] performs matrix multiplication on coefficient-encoded ciphertexts immediately after the S2C transformation, thereby leveraging the low-level coefficient representation for efficiency. However, unlike our approach, its linear transformation remains a standalone step rather than being fused into the bootstrapping pipeline.

Interactive secure transformer inference. Secure transformer inference protocols are typically based on a combination of FHE and secure two-party computation (2PC), which introduces extensive communication overhead between parties. Iron [25] optimizes Cheetah [27], a secure CNN protocol, by employing a more efficient packing strategy to reduce the cost of matrix multiplication. Bumblebee [35] further refines this packing strategy. BOLT [38] improves efficiency by combining optimized polynomial approximations for non-linear functions with a packing-aware matrix multiplication protocol. CipherGPT [26] presents custom protocols for matrix multiplication, GELU, and top-k sampling. Nimbus [33] introduces a novel outer-product paradigm for 2PC matrix multiplication. THE-X [11] and MPCFormer [32] replace GELU and Softmax with a combination of ReLU and polynomials; SecFormer [36] redesigns Softmax and develops efficient protocols for GELU and LayerNorm, achieving 3.4–24.7% accuracy improvement over MPCFormer. Nevertheless, due to the large errors introduced by such approximations, the original model parameters become unusable, necessitating additional retraining. Privformer [1], Puma [19], and Sigma [22] are three-party protocols that introduce additional trust assumptions; Mosformer [14] provides the first maliciously secure three-party inference framework. In general, these interactive approaches suffer from high communication overhead and require continuous client presence during inference.

NISTI. NEXUS [42] pioneered the area by showing that transformer inference can be performed non-interactively using FHE, introducing novel primitives like SIMD ciphertext compression/decompression and SIMD slot folding, dramatically reducing communication (e.g., 372.5× less than BOLT) while enabling GPU acceleration. THOR [37] builds upon this by proposing efficient matrix multiplication algorithms based on diagonal-packing, alongside optimized iterative methods for Softmax and LayerNorm. MOAI [43] further refines performance by introducing a consistent packing strategy (column and diagonal) that avoids costly format conversions between layers, rotation-free algorithms for Softmax and LayerNorm, and interleaved packing to minimize rotations, resulting in a 52.8% reduction in evaluation time compared to THOR.

8. Conclusion

In this paper, we introduce a fundamentally new paradigm for NISTI that shifts the focus from minimizing bootstrapping frequency to maximizing the amount of computation fused into each bootstrapping operation. By embracing bootstrapping as a first-class computational primitive rather than an occasional necessity, we demonstrate that the communication and computation overheads can be substantially reduced.

References

[1] Yoshimasa Akimoto, Kazuto Fukuchi, Youhei Akimoto, and Jun Sakuma. Privformer: Privacy-preserving transformer with mpc. In *2023 IEEE 8th European Symposium on Security and Privacy (EuroSP)*, pages 392–410, 2023.

[2] Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General functional bootstrapping using ckks. In Yael Tauman Kalai and Seny F. Kamara, editors, *Advances in Cryptology – CRYPTO 2025*, pages 304–337, Cham, 2025. Springer Nature Switzerland.

[3] Youngjin Bae, Jung Hee Cheon, Guillaume Hanrot, Jai Hyun Park, and Damien Stehlé. Plaintext-ciphertext matrix multiplication and fhe bootstrapping: Fast and fused. In *Advances in Cryptology – CRYPTO 2024: 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2024, Proceedings, Part III*, page 387–421, Berlin, Heidelberg, 2024. Springer-Verlag.

[4] Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping small integers with ckks. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 330–360, Singapore, 2025. Springer Nature Singapore.

[5] Song Bian, Yunhao Fu, Ruiyu Shen, Haowen Pan, Anyu Wang, and Zhenyu Guan. High-precision functional bootstrapping for ckks from fourier extension. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology – EUROCRYPT 2026*, pages 274–303, Cham, 2026. Springer Nature Switzerland.

[6] Marcelo Blatt, Alexander Gusev, Yuriy Polyakov, Kurt Rohloff, and Vinod Vaikuntanathan. Optimized homomorphic encryption solution for secure genome-wide association studies. Cryptology ePrint Archive, Paper 2019/223, 2019.

[7] Fabian Boemer, Anamaria Costache, Rosario Cammarota, and Casimir Wierzynski. ngraph-he2: A high-throughput framework for neural network inference on encrypted data. In *Proceedings of the 7th ACM Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, WAHC'19, page 45–56, New York, NY, USA, 2019. Association for Computing Machinery.

[8] Hyunho Cha, Intak Hwang, Seonhong Min, Jinyeong Seo, and Yongsoo Song. Matrigear: Accelerating authenticated matrix triple generation with scalable prime fields via optimized he packing. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 2453–2471, 2025.

[9] Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved bootstrapping for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 34–54. Springer, 2019.

[10] Hao Chen, Miran Kim, Ilya Razenshteyn, Dragos Rotaru, Yongsoo Song, and Sameer Wagh. Maliciously secure matrix multiplication with applications to private deep learning. In *Advances in Cryptology – ASIACRYPT 2020: 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part III*, page 31–59, Berlin, Heidelberg, 2020. Springer-Verlag.

[11] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. THE-X: Privacy-preserving transformer inference with homomorphic encryption. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3510–3520, Dublin, Ireland, May 2022. Association for Computational Linguistics.

[12] E. W. Cheney. *Introduction to Approximation Theory*. McGraw-Hill Book Company, New York, NY, USA, 1966.

[13] Elliott Ward Cheney and William Allan Light. *A course in approximation theory*, volume 101. American Mathematical Soc., 2009.

[14] Ke Cheng, Yuheng Xia, Anxiao Song, Jiaxuan Fu, Wenjie Qu, Yulong Shen, and Jiaheng Zhang. Mosformer: Maliciously secure three-party inference framework for large transformers. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 4124–4138, New York, NY, USA, 2025. Association for Computing Machinery.

[15] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018*, pages 360–384, Cham, 2018. Springer International Publishing.

[16] Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A full rns variant of approximate homomorphic encryption. In *Selected Areas in Cryptography–SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25*, pages 347–368. Springer, 2019.

[17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *Advances in Cryptology–ASIACRYPT 2017: 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I 23*, pages 409–437. Springer, 2017.

[18] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.*, 33(1):34–91, 2020.

[19] Ye Dong, Wen-jie Lu, Yancheng Zheng, Haoqi Wu, Derun Zhao, Jin Tan, Zhicong Huang, Cheng Hong, Tao Wei, and Wenguang Cheng. Puma: Secure inference of llama-7b in five minutes. *arXiv preprint arXiv:2307.12533*, 2023.

[20] Léo Ducas and Daniele Micciancio. Fhew: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015*, pages 617–640, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[21] Craig Gentry. *A fully homomorphic encryption scheme*. Stanford university, 2009.

[22] Kanav Gupta, Neha Jawalkar, Ananta Mukherjee, Nishanth Chandran, Divya Gupta, Ashish Panwar, and Rahul Sharma. Sigma: secure gpt inference with function secret sharing. *Cryptology ePrint Archive*, 2023.

[23] Shai Halevi and Victor Shoup. Algorithms in helib. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014*, pages 554–571, Berlin, Heidelberg, 2014. Springer Berlin Heidelberg.

[24] Shai Halevi and Victor Shoup. Faster homomorphic linear transformations in helib. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 93–120, Cham, 2018. Springer International Publishing.

[25] Meng Hao, Hongwei Li, Hanxiao Chen, Pengzhi Xing, Guowen Xu, and Tianwei Zhang. Iron: Private inference on transformers. *Advances in Neural Information Processing Systems*, 35:15718–15731, 2022.

[26] Xiaoyang Hou, Jian Liu, Jingyu Li, Yuhan Li, Jiawen Zhang, Wen-jie Lu, Cheng Hong, and Kui Ren. CipherGPT: Secure Two-Party GPT Inference . *IEEE Transactions on Dependable and Secure Computing*, 23(03):6748–6762, May 2026.

[27] Zhicong Huang, Wen-jie Lu, Cheng Hong, and Jiansheng Ding. Cheetah: Lean and fast secure {two-party} deep neural network inference. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 809–826, 2022.

[28] Ilia Iliashenko and Vincent Zucca. Faster homomorphic comparison operations for BGV and BFV. *Proceedings on Privacy Enhancing Technologies*, 2021(3):246–264, 2021.

[29] Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. Neujeans: Private neural network inference with joint optimization of convolution and fhe bootstrapping. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 4361–4375, New York, NY, USA, 2024. Association for Computing Machinery.

[30] Miran Kim, Yongsoo Song, Baiyu Li, and Daniele Micciancio. Semi-parallel logistic regression for GWAS on encrypted data. *IACR Cryptol. ePrint Arch.*, 2019:294, 2019.

[31] Joon-Woo Lee, Eunsang Lee, Yongwoo Lee, Young-Sik Kim, and Jong-Seon No. High-precision bootstrapping of rns-ckks homomorphic encryption using optimal minimax polynomial approximation and inverse sine function. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021*, pages 618–647, Cham, 2021. Springer International Publishing.

[32] Dacheng Li, Rulin Shao, Hongyi Wang, Han Guo, Eric P Xing, and Hao Zhang. Mpcformer: fast, performant and private transformer inference with mpc. *International Conference on Learning Representations (ICLR)*, 2023.

[33] Zhengyi Li, Kang Yang, Jin Tan, Wen jie Lu, Haoqi Wu, Xiao Wang, Yu Yu, Derun Zhao, Yancheng Zheng, Minyi Guo, and Jingwen Leng. Nimbus: Secure and efficient two-party inference for transformers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[34] Zeyu Liu and Yunhao Wang. Amortized functional bootstrapping in less than 7 ms, with $\tilde{O}(1)$ polynomial multiplications. In Jian Guo and Ron Steinfeld, editors, *Advances in Cryptology – ASIACRYPT 2023*, pages 101–132, Singapore, 2023. Springer Nature Singapore.

[35] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Kui Ren, Cheng Hong, Tao Wei, and WenGuang Chen. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive*, 2023.

[36] Jinglong Luo, Yehong Zhang, Zhuo Zhang, Jiaqi Zhang, Xin Mu, Hui Wang, Yue Yu, and Zenglin Xu. Secformer: Fast and accurate privacy-preserving inference for transformer models via smpc, 2025.

[37] Jungho Moon, Dongwoo Yoo, Xiaoqian Jiang, and Miran Kim. Thor: Secure transformer inference with homomorphic encryption. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 3765–3779, New York, NY, USA, 2025. Association for Computing Machinery.

[38] Qi Pang, Jinhao Zhu, Helen Möllering, Wenting Zheng, and Thomas Schneider. Bolt: Privacy-preserving, accurate and efficient inference for transformers. *IEEE Symposium on Security and Privacy (SP)*, 2024.

[39] Walter Rudin. *Principles of Mathematical Analysis*. McGraw-Hill Book Company, New York, NY, USA, 3rd edition, 1976.

[40] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. In *International Conference on Learning Representations*, 2019.

[41] Zhaomin Yang, Chao Niu, Benqiang Wei, Zhicong Huang, Cheng Hong, and Tao Wei. RBOOT: Accelerating homomorphic neural network inference by fusing ReLU within bootstrapping. Cryptology ePrint Archive, Paper 2025/1534, 2025.

[42] Jiawen Zhang, Xinpeng Yang, Lipeng He, Kejia Chen, Wen-jie Lu, Yinghao Wang, Xiaoyang Hou, Jian Liu, Kui Ren, and Xiaohu Yang. Secure transformer inference made non-interactive. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*, New York, NY, USA, 2025. The Internet Society.

[43] Linru Zhang, Xiangning Wang, Jun Jie Sim, Zhicong Huang, Jiahao Zhong, Huaxiong Wang, Pu Duan, and Kwok-Yan Lam. MOAI: Module-optimizing architecture for non-interactive secure transformer inference. In *The Fourteenth International Conference on Learning Representations*, 2026.

[44] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 14901–14911, June 2025.

Appendix A. Homomorphic Matrix Multiplication

We show how different packing formats can be multiplied with each other.

- *row-packing* $\times$ *plaintext* $\rightarrow$ *row-packing* [24]. Let $\widetilde{\mathbf{X}}^{\mathsf{row}}$ be the row-packing of $\mathbf{X} \in \mathbb{R}^{m\times n}$. To compute $\mathbf{Y} = \mathbf{X}\cdot\mathbf{W}$, where $\mathbf{W} \in \mathbb{R}^{n\times d}$, $\forall i \in [m]$, the i-th row of $\mathbf{Y}$ can be computed using the *Baby-Step-Giant-Step* (BSGS) [24] strategy as follows:

$$\begin{aligned}\widetilde{\mathbf{Y}}^{\mathsf{row}}[i] &= \sum_{j=0}^{d-1} \mathsf{Rot}_j\left(\widetilde{\mathbf{X}}[i]\cdot \mathsf{diag}_{-j}(\mathbf{W})\right)\\ &= \sum_{j_1=0}^{\sqrt{d}-1}\sum_{j_2=0}^{\sqrt{d}-1} \mathsf{Rot}_{j_1\cdot\sqrt{d}+j_2}\left(\widetilde{\mathbf{X}}[i]\cdot \mathsf{diag}_{-j_1\cdot\sqrt{d}-j_2}(\mathbf{W})\right)\\ &= \sum_{j_1=0}^{\sqrt{d}-1} \mathsf{Rot}_{j_1\cdot\sqrt{d}}\sum_{j_2=0}^{\sqrt{d}-1} \mathsf{Rot}_{j_2}\left(\widetilde{\mathbf{X}}[i]\cdot \mathsf{diag}_{-j_1\cdot\sqrt{d}-j_2}(\mathbf{W})\right)\end{aligned}$$

This method requires a total of md ciphertext-plaintext multiplications and $2m\sqrt{d}$ rotations. For simplicity, we denote it as:

$$\widetilde{\mathbf{Y}}^{\mathsf{row}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{row}}\cdot\mathbf{W}.$$

- *column-packing* $\times$ *plaintext* $\rightarrow$ *column-packing* [23]. Let $\widetilde{\mathbf{X}}^{\mathsf{col}}$ be the column-packing of $\mathbf{X} \in \mathbb{R}^{n\times m}$. To compute $\mathbf{Y} = \mathbf{X}\cdot\mathbf{W}$, where $\mathbf{W} \in \mathbb{R}^{m\times d}$, $\forall i \in [d]$, the i-th column of $\mathbf{Y}$ can be computed as follows:

$$\widetilde{\mathbf{Y}}^{\mathsf{col}}[i] = \sum_{j=0}^{m-1} \widetilde{\mathbf{X}}^{\mathsf{col}}[j]\cdot\mathbf{W}[j,i].$$

This method requires $d\cdot m$ ciphertext-plaintext multiplications. We denote it as:

$$\widetilde{\mathbf{Y}}^{\mathsf{col}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{col}}\cdot\mathbf{W}.$$

- *column-packing* $\times$ *row-packing* $\rightarrow$ *diagonal-packing* [38]. Let $\widetilde{\mathbf{X}}^{\mathsf{col}}$ be the column-packing of $\mathbf{X} \in \mathbb{R}^{n\times d}$ and $\widetilde{\mathbf{Y}}^{\mathsf{row}}$ be the row-packing of $\mathbf{Y} \in \mathbb{R}^{d\times n}$. To compute $\mathbf{Z} = \mathbf{X}\cdot\mathbf{Y}$, $\forall i \in [n]$, the i-th diagonal of $\mathbf{Z}$ can be computed as:

$$\widetilde{\mathbf{Z}}^{\mathsf{diag}}[i] = \sum_{j=0}^{d-1} \widetilde{\mathbf{X}}^{\mathsf{col}}[j]\cdot \mathsf{Rot}_i(\widetilde{\mathbf{Y}}^{\mathsf{row}}[j]).$$

This method requires nd ciphertext-ciphertext multiplications and nd rotations. We denote it as:

$$\widetilde{\mathbf{Z}}^{\mathsf{diag}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{col}}\cdot\widetilde{\mathbf{Y}}^{\mathsf{row}}.$$

- *diagonal-packing* $\times$ *column-packing* $\rightarrow$ *column-packing* [23]. Let $\widetilde{\mathbf{X}}^{\mathsf{diag}}$ be the diagonal-packing of $\mathbf{X} \in \mathbb{R}^{n\times n}$ and $\widetilde{\mathbf{Y}}^{\mathsf{col}}$ be the column-packing of $\mathbf{Y} \in \mathbb{R}^{n\times d}$. To compute $\mathbf{Z} = \mathbf{X}\cdot\mathbf{Y}$, $\forall i \in [d]$, the i-th column of $\mathbf{Z}$ can be computed as:

$$\widetilde{\mathbf{Z}}^{\mathsf{col}}[i] = \sum_{j=0}^{n-1} \widetilde{\mathbf{X}}^{\mathsf{diag}}[j]\cdot \mathsf{Rot}_j(\widetilde{\mathbf{Y}}^{\mathsf{col}}[i]).$$

This procedure can also be optimized by BSGS, and requires nd ciphertext-ciphertext multiplications and $2d\sqrt{n}$ rotations. We denote it as:

$$\widetilde{\mathbf{Z}}^{\mathsf{col}} \leftarrow \widetilde{\mathbf{X}}^{\mathsf{diag}}\cdot\widetilde{\mathbf{Y}}^{\mathsf{col}}.$$

Appendix B.
Proof for Lemma 3.1

Proof. By definition, each $\theta \in \mathbb{R}^k$ determines a trigonometric polynomial $p_\theta \in \mathcal{T}_d$. Furthermore, each $p \in \mathcal{T}_d$ can be written in the form

$$p(x) = c_0 + \sum_{m=1}^{d} \big(a_m \cos(m\eta x) + b_m \sin(m\eta x)\big)$$

for a certain coefficient vector

$$\theta = (c_0, a_1, b_1, \ldots, a_d, b_d) \in \mathbb{R}^k.$$

Therefore, the map ξ is onto.

It remains to show that ξ is one-to-one. Suppose that two coefficient vectors $\theta, \theta' \in \mathbb{R}^k$ define the same trigonometric polynomial:

$$p_\theta(x) = p_{\theta'}(x), \qquad \forall x \in [\alpha, \beta].$$

Then,

$$p_{\theta-\theta'}(x) = 0, \qquad \forall x \in [\alpha, \beta].$$

Choose k distinct points

$$x_1, \ldots, x_k \in [\alpha, \beta].$$

Let

$$\{c_1, \ldots, c_k\} = \{1, \cos\eta x, \sin\eta x, \ldots, \cos(d\eta x), \sin(d\eta x)\}$$

and

$$\mathbf{Y} = \big(c_j(x_i)\big)_{i,j=1}^{k}.$$

Since $\{c_1, \ldots, c_k\}$ satisfies the Haar condition on $[\alpha, \beta]$, the matrix $\mathbf{Y}$ is invertible. Evaluating $p_{\theta-\theta'}$ at $x_1, \ldots, x_k$ gives

$$\big(p_{\theta-\theta'}(x_1), \ldots, p_{\theta-\theta'}(x_k)\big)^\top = \mathbf{Y} \cdot (\theta - \theta')^\top = [0, \ldots 0]^\top.$$

Since $\mathbf{Y}$ is invertible, we obtain

$$\theta - \theta' = [0, \ldots 0].$$

Therefore,

$$\theta = \theta',$$

and ξ is one-to-one.

Since ξ is both onto and one-to-one, it is a bijection. Thus, searching for the *degree-d trigonometric minimax approximation* of f_T over $\mathcal{T}_d$ is equivalent to searching for the coefficient vector over $\mathbb{R}^k$. □